

Project Title: Online Bookstore

Planning Phase

Team Member:

Name	Student ID	Task
Mohammad Poorahmadi	2309015528	Planning Phase & Development Phase
Syed Hussain Razavi	2309015120	Planning Phase
Praise-God Tobby	2309116418	Requirement Phase
Abdulrahman Bin Omran	2309015473	Designing Phase
Ehsanullah Enayatullah	229913876	Testing Phase
Ahmet Masri	2309116264	Deployment Phase

Problem statement

Many readers struggle to find a convenient way to discover, compare and purchase books with reliable search, clear inventory status, secure checkout and order tracking. There is a need for a centralized, automated, user-friendly system where customers can easily explore and purchase books online while allowing administrators to maintain accurate data & offer secure online payments and automated order tracking. The cost must be within the budget, The system must be fully functional mobile application with online payment support and support cloud storage. This application should be operational by 24 December, 2025.

Vision

The vision of the online bookstore mobile application is to create an easy to use, and efficient platform where users can browse, purchase and read books conveniently from their mobile devices. The system aims to simplify the book buying experience, support digital reading and provide personalized environment for book lovers.

Scopes

IN Scope:

- User registration and login
- browsing books by category, author or rating
- Viewing detailed book information (Price, summary, reviews)
- adding books to cart and purchasing
- Integrated secure payment
- Order history

- Admin panel(add/edit/delete books, manage orders and accounts)
- digital library for purchased ebooks
- Search functionality
- Notifications on new arrival and promotions

Out Scope:

- Audiobooks
- Social features(community, user to user chat)
- advanced recommendation engine
- Multi language UI

High Level User Requirements

Functional:

- Customer can create account and login.
- Customer can browse, search and view books.
- Customer must be able to add books to the shopping cart.
- Customer can purchase book securely.
- Admin can manage books, orders and accounts.
- Customer can access purchased ebook.
- The system must notify users of successful orders.

Non Functional:

- The interface must be mobile friendly and fast.
- The system must be scalable to support future features.

Initial feasibility analysis

This is an early stage to check whether an ebook store is technically doable, financially viable, legally safe and operationally realistic.

1. Market Feasibility = Piracy, Books being shared, not enough catalog.
2. Product / Technical Feasibility = Website security can be compromised & not have enough catalog or inventory
3. Admin / Employee access = Not enough trained staff or the cost of experienced employee is very high
4. Legal / Copyright Feasibility = This is sensitive part of an ebook store, should have distribution rights, duration, revenue share, copyright complaints, term of service & privacy policy.
5. Major Early Risks = Competing with big platforms or payment frictions.

Phases activities tasks and deliverables

Phase 1 Planning:

Activities: Define vision and scope, Problem statement, initial feasibility analysis.

Deliverables : Project proposal, time line, risk register & cost estimate, SPMP.

Phase 2 Requirements:

Activities :The target audience, technical feasibility, MVP criteria

Deliverables : Software specification, User stories + backlog & wireframe requirements

Phase 3 System Design

Activities :Architecture design (Data base, website & services), the security & access model, the data model & API contracts

Deliverables : High level design, low level design, database schema & API documentation, UI/UX wireframes or mockups

Phase 4 Development

Activities: Implement mobile frontend, Develop backend APIs, Integrate payment system, Create admin panel, Storage setup

Deliverables : Fronted web apps, backend APIs, Admin panel, payment integration, storage

Phase 5 Testing

Activities : Functional testing, Security Testing, Performance smoke test & User acceptance testing

Deliverables: Test plane, Test cases, Test report, UAT result.

Phase 6 Deployment

Activities: Set up production environment, Configure CI/CD pipeline,Implement monitoring and logging, prepare release notes and rollback plan, Final verification and go live

Deliverables : Production hosting setup, CI/CD pipeline, Domain, SSL, monitoring, Release notes + rollback plan.

Resources

Amazone cloud services

SRS with scope, PDF.js(PDF reader [mozilla.github.io](https://mozilla.github.io/pdf.js/)), epub.js (EPUB reader github)

Payment integration

A full verification standard that can be as a security requirements checklist (auth, access control, session management, data protection, etc.)

Schedule Estimations

3 Weeks

1st week -> planning and requirements

2nd week -> System design & Development

3rd week -> testing & deployment

Cost estimations

\$50,000 to \$100,000

Better search / Filters, coupons / promotions, richer reader features, deeper reporting,
stronger security hardening

In planning phase

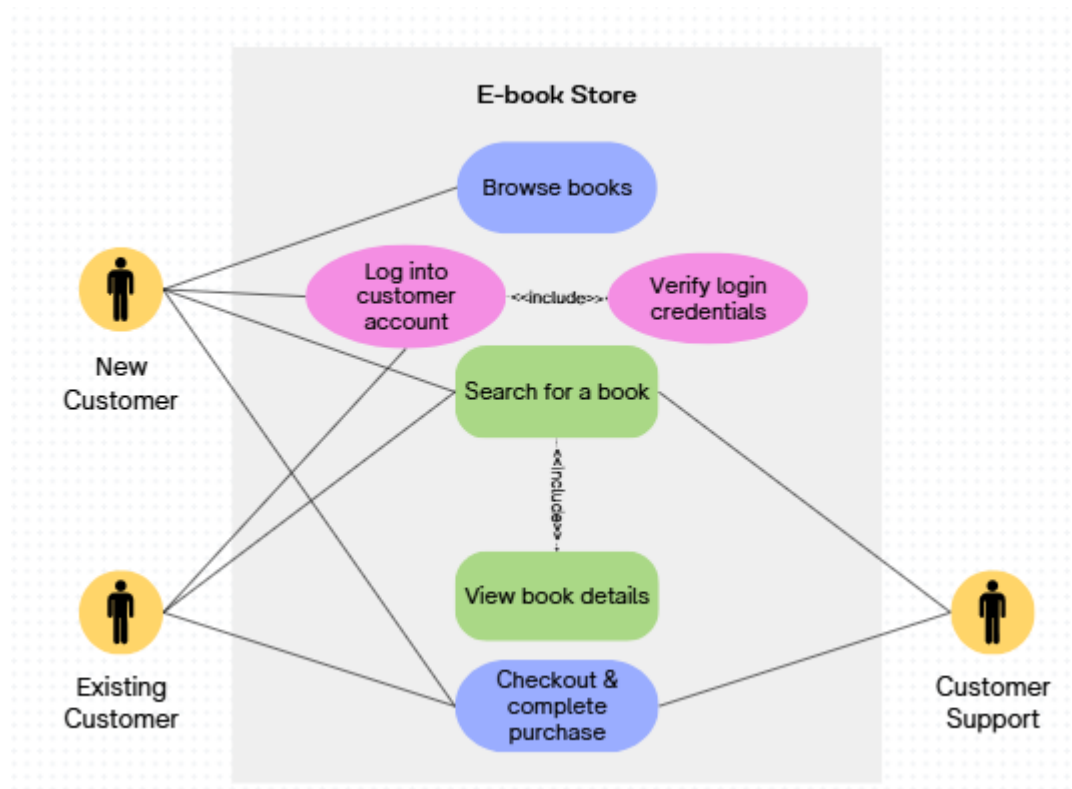
MVP budget \$20k - \$35k

Timeline = 3 Weeks

Maintenance = \$500-\$3000 / Month depending on pace

Requirements Phase

UML DIAGRAM



Functional Requirements

1. Customer Management

Requirement:

Customers should be able to create accounts and securely access their profiles.

Scope:

The system will provide a secure registration and authentication module for signing up, logging in, and logging out.

2. Employee Operations

Requirement:

Employees should be able to maintain and update the e-book catalog efficiently.

Scope:

Employees will have controlled administrative access allowing them to add new books, update book details, and remove outdated titles, while being restricted from

sensitive admin-only features such as handling payments or accessing full system settings.

3. E-Book Browsing & Search

Requirement:

Users should be able to explore and locate e-books using filters like title, author, and genre.

Scope:

The platform will include a fast, accurate search engine and a user-friendly browsing interface.

4. Book Details Display

Requirement:

Users should be able to view comprehensive information about each book, including ratings and reviews.

Scope:

Each book page will display title, author, price, description, ratings, and user reviews.

5. Shopping Cart Management

Requirement:

Users should be able to manage their shopping cart easily.

Scope:

A dynamic cart module will support adding, removing, and updating book quantities, with automatic total price calculation.

6. Payment & E-Book Access

Requirement:

Users should be able to purchase e-books securely and access them instantly.

Scope:

A secure checkout system, card storage option, instant download, and a Library/My Books page will be provided.

7. Administrator Panel

Requirement:

Administrators should be able to manage catalog content and customer orders.

Scope:

An admin console will enable adding, editing, deleting e-books, and managing all customer orders.

8. User Reviews & Ratings

Requirement:

Users should be able to share feedback by posting reviews and rating purchased e-books.

Scope:

The system will allow verified buyers to submit ratings, write reviews, edit or delete their feedback, and view community reviews on each book page.

9. Order History & Email Notifications

Requirement:

Users should be able to track their past purchases and receive important order-related updates.

Scope:

The platform will provide an order history page displaying all completed purchases and will automatically send email notifications for events such as successful payments, account changes, and password resets.

Non-Functional Requirements

1. Performance

Requirement:

The system should provide fast response times and operate smoothly under heavy traffic.

Scope:

Optimized servers, caching, and efficient back-end logic will ensure high performance.

2. Security

Requirement:

User and platform data should be fully protected.

Scope:

Strong encryption, secure authentication, and strict access control will secure all sensitive information.

3. Reliability

Requirement:

The system should be consistently available and safeguard user purchases.

Scope:

Redundant systems, backups, and stable infrastructure will ensure continuous access.

4. Usability

Requirement:

The interface should be intuitive and easy to navigate across all devices.

Scope:

A responsive, clean, and user-friendly UI will deliver a seamless user experience.

5. Maintainability

Requirement:

The system should be easy to update, extend, and improve over time.

Scope:

A modular architecture and clean codebase will support efficient maintenance and future enhancements.

6. Scalability

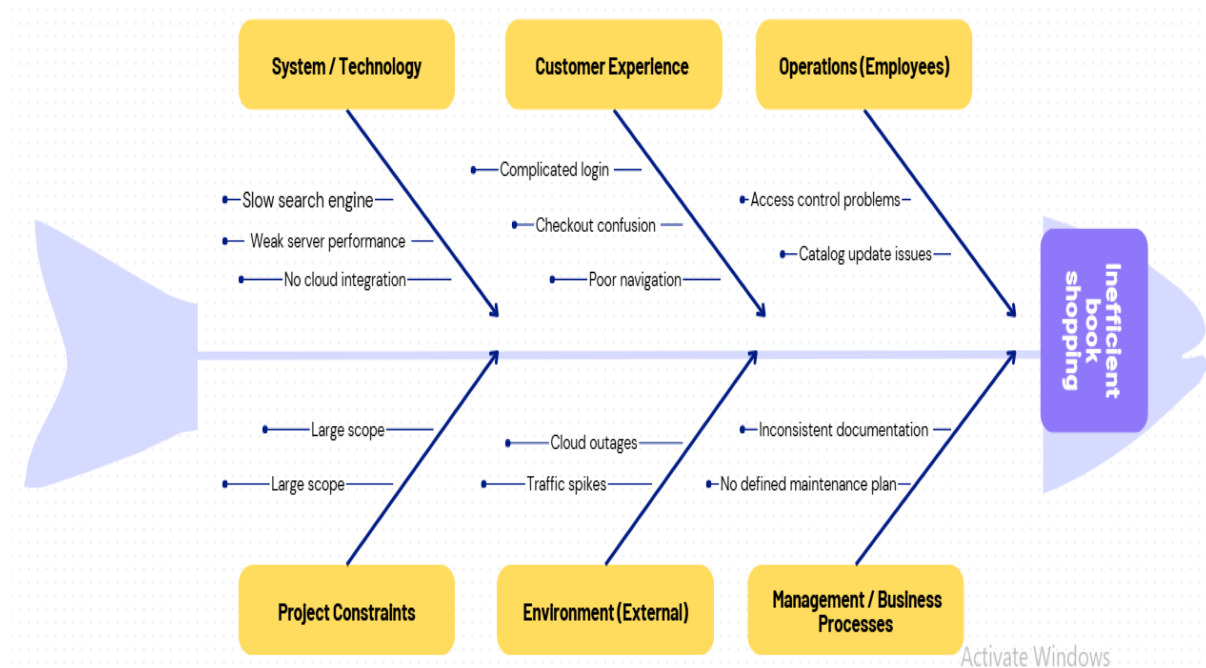
Requirement:

The system should be able to grow and support increased users, books, and transactions without reducing performance.

Scope:

The platform will be built with scalable infrastructure that allows adding more resources, expanding storage, and handling higher traffic as the user base grows.

FISHBONE DIAGRAM



Fact-Finding Techniques

1. Research / Documentation Review:

Reviewing existing documents, reports, websites, and system manuals to understand how similar systems work.

Example:

Researching popular online bookstores (e.g., Amazon Books, Kobo) to understand how they handle book search, recommendations, and checkout flows.

2. Observation of the Work Environment:

Watching real users interact with existing systems or processes to identify pain points and inefficiencies.

example:

Observing how customers currently struggle to find books on scattered websites, take too long comparing prices, and experience confusion during checkout.

3. Interviews:

Speaking directly with users or stakeholders to gather detailed insights about their needs and frustrations.

Example:

Interviewing book buyers who say they get confused by unreliable search results and want clearer inventory information before purchasing.

4. Questionnaires / Surveys:

Bringing stakeholders together in a workshop to collaboratively define system requirements.

Example:

Holding a session with customers, bookstore staff, and system developers to agree on must-have features like category filters, book previews, and an automated order tracking dashboard.

5. Prototyping:

Creating a preliminary version of the system to gather user feedback early.

Example:

Building a quick mockup of the search page with filters to test whether users find it easier to locate books.

Use Case Modelling

Use Case Modelling describes how users interact with the system to achieve goals.

It focuses on actors, actions, and expected results.

Purpose of Use Case Modelling

Use cases help define clear functional requirements by illustrating what the system should do for its users.

Example:

A “Search for Books” use case shows how a customer enters keywords, applies filters, and receives book results sorted by relevance.

Primary Actors:

Customer – the person who browses, searches, and purchases books.

Admin – manages system settings, users, and high-level controls.

Employee (Catalog Manager) – maintains, updates, and organizes the book catalog.

Secondary Actors:

Payment Gateway – processes customer payments securely.

System (Automated Processes) – sends order updates, confirmations, and performs background tasks.

Use Case: Purchase a Book

Actor: Customer

Preconditions: Customer is logged in.

Main Flow:

1. Customer searches for a book.
2. Customer selects a book and views details.
3. Customer adds the book to cart.
4. Customer proceeds to checkout.
5. Customer enters payment details and confirms purchase.

Postconditions:

- Customer receives an order confirmation.
- System updates order tracking status and inventory.

Use Case: Update Book Catalog

Actor: Employee(Catalog Manager)

Preconditions: Catalog Manager is logged into the admin panel.

Main Flow:

1. Catalog Manager opens the book catalog management page.
2. System displays existing book entries.
3. Catalog Manager edits, adds, or deletes book information.
4. System validates and saves the changes.

Postconditions: The updated book catalog is stored in the system.

Use Case: Process Payment

Actor: Payment Gateway

Preconditions: Customer has initiated checkout.

Main Flow:

1. System sends payment details to the Payment Gateway.
2. Payment Gateway validates card or wallet information.

3. Payment Gateway approves or rejects the transaction.
4. System receives the payment status.

Postconditions: A transaction result is saved and linked to the order.

Use Case: Manage User Accounts

Actor: Admin

Preconditions: Admin is authenticated into the admin dashboard.

Main Flow:

1. Admin opens the user account management panel.
2. System displays the full list of customers.
3. Admin views, edits, or disables specific user accounts.
4. System saves all changes.

Postconditions: User account modifications are successfully updated in the system.

USER STORY

INITIATIVE	EPIC	USER STORY	ACCEPTANCE CRITERIA
Develop an online book store and purchase system	As a customer, I want to access the online book platform behind a secure login so that I can discover and purchase books.	As a customer, I want to search for books easily so that I can find titles that match my interests.	Ensure the customer is able to: - Log in securely - Use a search bar to find books by title/author/genre - View accurate search results
	As a customer, I want clear book details so that I can compare options and make informed choices.	As a customer, I want to view detailed book information so I can compare books before buying.	Ensure the customer is able to: -View book summaries, prices, and availability -Compare multiple books -See reviews or ratings (if applicable)

	As a customer, I want a smooth checkout process so that I can complete my purchase without issues.	As a customer, I want to purchase books securely so I can receive them without worry.	Ensure the customer is able to: - Add books to cart - Complete secure checkout - Receive order confirmation
	As a customer, I want to track my orders so that I know when to expect my purchase.	As a customer, I want to view my order status so that I can follow its progress.	Ensure the customer is able to: - Access their order history - View order tracking updates

Designing Phase

1. Introduction

1.1 Overview

The goal of this phase is to transform the user and business requirements gathered in the previous **Requirements Analysis Phase** into a technical blueprint (Software Design Document) that developers can implement.

1.2 Inputs (From Requirements Phase)

Based on the **Requirements Document**, we are designing for the following core functionalities:

- **Customer Module:** Registration, Login, Profile Management.
- **Catalog Module:** Search, Browse, View Details.
- **Commerce Module:** Shopping Cart, Checkout, Payment Integration.
- **Admin Module:** User Management, Catalog Updates.

1.3 Scope of the Design Phase

According to the provided course materials, this phase focuses on defining the internal structure of the software. We will cover two levels of design:

1. **Architectural Design (System Design):** Defining subsystems, module interactions, and database structures.
2. **Detailed Design:** Specifying algorithms, data structures, and internal logic for components.

2. Development: System Design

In this section, we translate the "Functional Requirements" into "Technical Models" using UML diagrams as the primary modeling tool.

2.1 Software Architecture (High-Level Design)

We will use a **Layered Architecture** to separate concerns, ensuring the "Maintainability" requirement is met.

- **Presentation Layer:** Web Interface (React/HTML5) for Customers and Admins.
- **Business Logic Layer:** Services for Authentication, Catalog Management, and Order Processing.
- **Data Access Layer:** APIs communicating with the database.
- **Database Layer:** SQL Database storing Users, Books, and Orders.

2.2 Structural Design (UML Class Diagram)

Based on the entities identified in the **Requirements** (Customer, Book, Order, Employee), we define the following classes and relationships:

- **Class: User**
 - *Attributes:* `userId`, `email`, `passwordHash`, `role`
 - *Methods:* `login()`, `logout()`, `updateProfile()`
 - *Subclasses:* `Customer` (adds `address`, `paymentMethods`), `Administrator` (adds `manageUsers()`).
- **Class: Book**
 - *Attributes:* `isbn`, `title`, `author`, `genre`, `price`, `stockQuantity`, `reviews`
 - *Methods:* `updateDetails()`, `getAverageRating()`
 - *Relevance:* Supports the "Book Details Display" requirement.

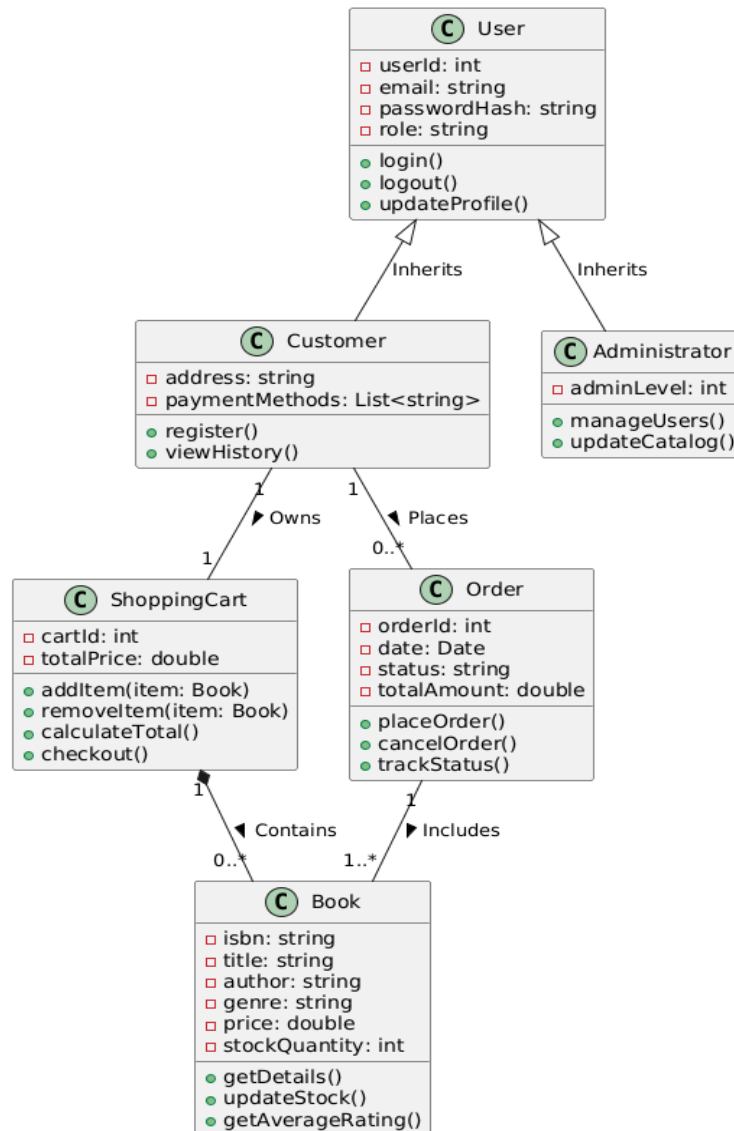
- **Class: ShoppingCart**

- *Attributes:* cartId, items (List of CartItem), totalPrice
- *Methods:* addItem(), removeItem(), calculateTotal().
- *Relevance:* Directly addresses "Shopping Cart Management" requirements.

- **Class: Order**

- *Attributes:* orderId, date, status, totalAmount
- *Methods:* placeOrder(), cancelOrder(), trackStatus()

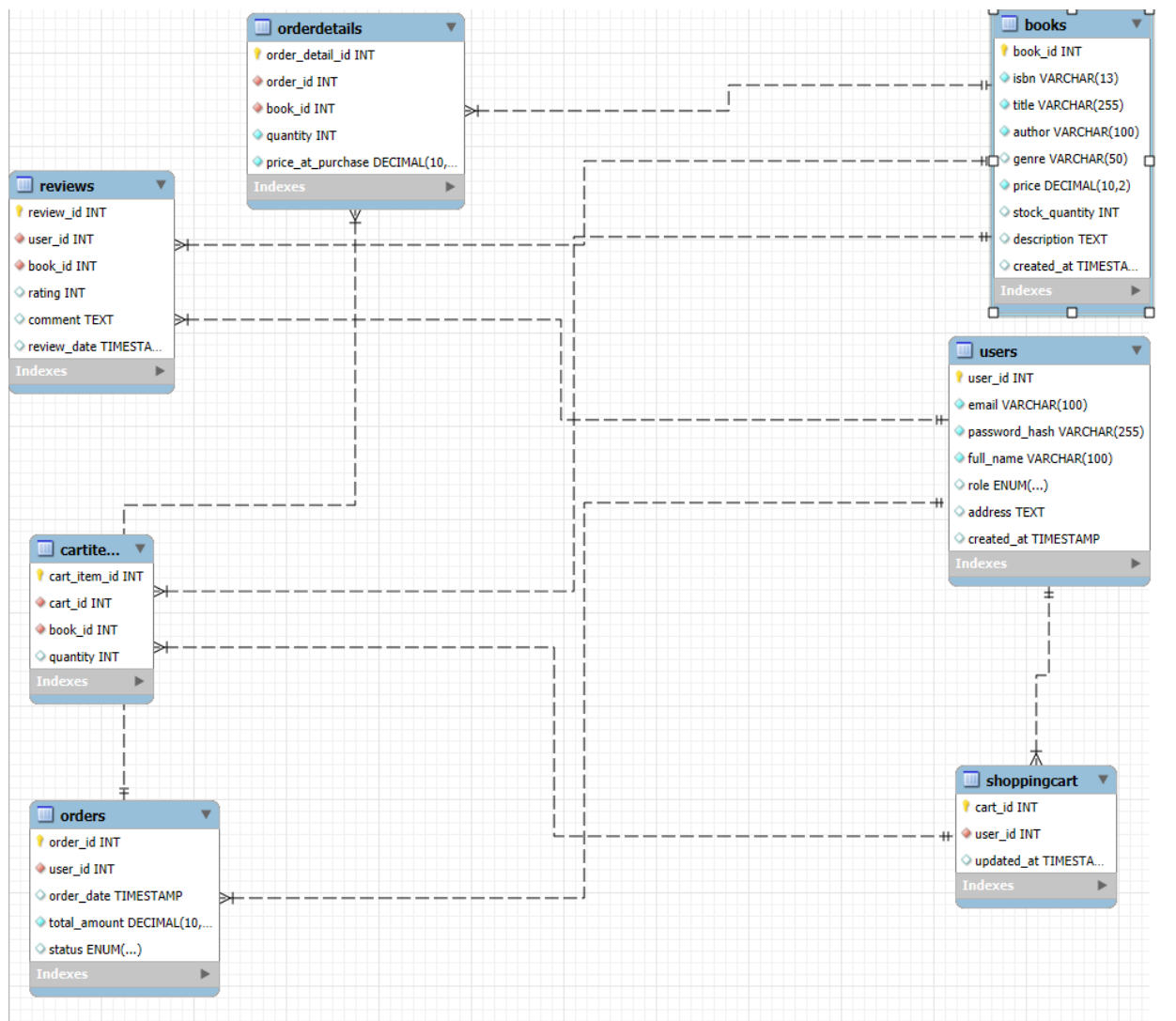
Online Bookstore System - Class Diagram



2.3 Database Design (ER Diagram Concept)

To support the "Data Storage" needs:

1. **Table Users:** 1:N relationship with Orders and Reviews.
2. **Table Books:** 1:N relationship with Reviews; N:M relationship with Orders (via OrderDetails table).
3. **Table Transactions:** Linked to Orders to track payment status (Success/Fail).

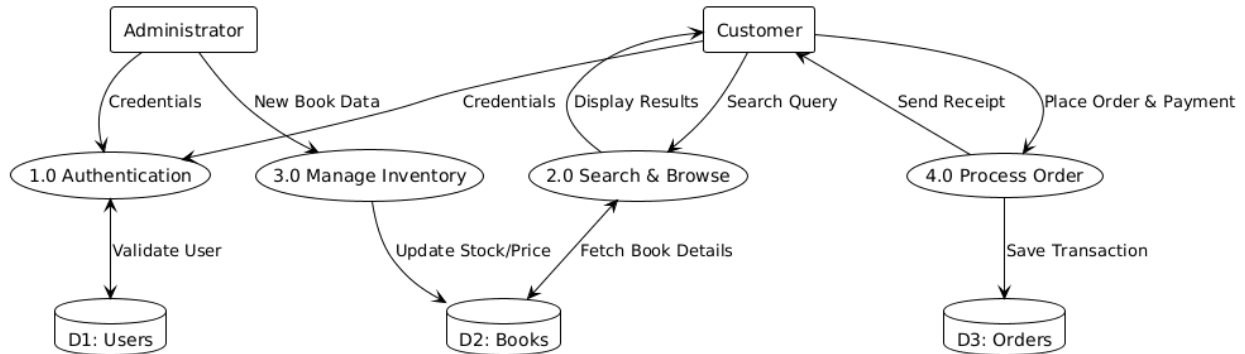


2.4 Component Design

We identify the key subsystems and their interfaces:

- **SearchEngine Component:** Provides `ISearch` interface. Depends on Inventory database.
- **PaymentGateway Component:** External system integration. Requires `processPayment(amount, currency)` interface.
- **NotificationService Component:** Handles email alerts for "Order History & Email Notifications" requirement.

Data Flow Diagram (DFD) - Level 1



3. Development: Detailed Design

This section details the logic and flow of specific features using Behavioral Diagrams.

3.1 Use Case Logic: "Purchase a Book" (Sequence Diagram)

We map the flow for the **Purchase a Book** use case found in the Requirements file:

1. **Actor (Customer)** clicks "Checkout".
2. **UI** sends `initiateCheckout()` message to **OrderService**.
3. **OrderService** calls `InventorySystem` to `checkStock(bookId)`.
4. **Alt Frame (Condition):**
 - a. *If In Stock:* **OrderService** calls **PaymentGateway** `processPayment()`.
 - b. *If Out of Stock:* Return error "Item unavailable".
5. **PaymentGateway** returns `Success`.
6. **OrderService** creates `Order` object and calls **EmailService** to send confirmation.
7. **UI** displays "Order Confirmed".

3.2 Process Flow: "User Login" (Activity Diagram)

To satisfy the "Security" requirement for secure access:

- **Start Node:** User enters credentials.
- **Action:** System hashes password.
- **Decision Node:** Compare hash with Database.
 - *Match:* **Action:** Generate Session Token -> **End Node:** Redirect to Homepage.
 - *No Match:* **Action:** Increment retry count -> **Decision:** Retry < 3?
 - *Yes:* Show "Invalid Credentials".
 - *No:* Lock Account (Security Requirement).

4. Conclusion

4.1 Deliverables Produced

In this phase, we have successfully delivered the **Software Design Document (SDD)**, which includes:

- **Architecture Diagrams:** High-level system structure.
- **Class Diagrams:** Data and object structure.
- **Sequence & Activity Diagrams:** Logic flow for critical requirements (Purchase, Login).
- **Database Schema:** Data organization.

Phase 4: Software Development Report

1. Executive Summary

The objective of the Development Phase was to translate the system design into a functional software product. In accordance with the project schedule, the application has been built as a Responsive Web Application that works on both mobile and desktop devices. The system successfully implements the core functional demo features: User Authentication, Inventory Management, Secure Ordering, and Administration.

2. Technical Architecture

The system follows the architecture defined in the Design Document, implemented using the following stack:

Frontend: HTML, CSS

Backend Logic: JavaScript

Database: Browser LocalStorage

Security Standard: Session-based authentication with account locking policies.

3. Implementation of Deliverables

3.1 Frontend Web Application

The user interface was developed to be mobile-friendly and intuitive.

- **Login Module (login.html):** Implements the entry point for both Customers and Admins. Includes a toggle for "User Registration" that enforces data validation (e.g., Address requirement).
- **Storefront (store.html):** Displays the book catalog with real-time updates. Features include dynamic price calculation, star ratings based on reviews, and an interactive shopping cart.

3.2 Backend Logic & Storage Setup

- **Core Logic (app.js):** This file acts as the central controller, handling all business rules defined in the Sequence Diagrams.
- **Data Persistence:** We implemented a `localStorage` database simulation to satisfy the "Storage Setup" requirement. This allows user accounts, book inventory, and order history to be saved even after the browser is closed.
- **Inventory Logic:** The system automatically decrements stock levels upon purchase to prevent overselling, adhering to the "Clear Inventory Status" requirement.

3.3 Admin Panel Development

A dedicated dashboard (`admin.html`) was created to separate administrative privileges from customer access.

- **Capabilities:** Administrators can view all registered users and manage the inventory.
- **Dynamic Catalog:** The "Update Catalog" feature allows admins to add new products instantly without modifying the source code.

3.4 Payment & Security Integration

- **Security:** Implemented an "Account Lockout" mechanism that freezes an account after 3 failed login attempts, satisfying the security requirements.
- **Payment:** The `placeOrder()` function simulates a secure checkout process, validating stock availability and generating a unique Transaction ID for every order.

4. Code Structure

`app.js` – Contains the "Mock Database," Authentication Logic, and Order Services.

`login.html` – Handles User Registration, Login, and Security Alerts.

`store.html` – The main interface for browsing books, managing the cart, and viewing history.

admin.html – The secure dashboard for User Management and Inventory updates.

5. Challenges & Solutions

- **Challenge:** Implementing persistent data without a dedicated backend server.
- **Solution:** Utilized the browser's LocalStorage API to persist User, Book, and Order data, ensuring a seamless user experience during the demo.

6. Conclusion

The Development Phase is complete. The system is ready for Testing Phase.

Phase 5: Software Testing Report

1. Test Plan & Objective

The objective of this phase is to perform Functional Testing and Security Testing as defined in the project schedule. We aim to verify that the system meets the MVP criteria, including user registration, secure purchasing, and clear inventory status.

Test Environment:

- **Browser:** Safari
- **Database:** Client-Side LocalStorage
- **Application Files:** login.html, store.html, admin.html, app.js

2. Test Cases & Results

Module A: Registration & Security

Verifies requirements: User registration and login, Security policies.

Test ID	Test Case Description	Steps to Execute	Expected Result	Status
Test Case 1	Validate Address Requirement	Fill registration form without address and submit.	System displays error message requiring address.	PASS
Test Case 2	Account Security Lockout	Enter wrong password 3 times then correct password.	Account is locked with security alert.	PASS
Test Case 3	Successful Login	Register user and login with correct credentials.	Redirected to store with welcome message.	PASS

Module B: Inventory & Purchasing

Verifies requirements: Clear inventory status, Secure purchasing.

Test ID	Test Case Description	Steps to Execute	Expected Result	Status
Test Case 4	Stock Quantity Display	View catalog as customer.	Stock quantity displayed for each book.	PASS

Test Case 5	Prevent Out-of-Stock Order	Add more items than available stock.	Order blocked with out-of-stock error.	PASS
Test Case 6	Successful Purchase	Checkout with available stock.	Order confirmed and stock updated.	PASS
Test Case 7	Product Review System	Login as customer and Write Review for a book .	The book's average stare rating update immediately in the catalog view.	PASS

Module C: Administrator Functions

Verifies requirements: Admin panel and catalog management.

Test ID	Test Case Description	Steps to Execute	Expected Result	Status
Test Case 8	Admin Dashboard Access	Login as admin.	Redirected to admin panel.	PASS
Test Case 9	Update Catalog (Add Book)	Add new book via admin panel.	Book appears in customer catalog.	PASS
Test Case 10	Manage User Account	Login as admin and Click Manage Users tab.	Admin can view a list of all registered users, including their IDs, Emails and Addresses.	PASS

3. User Acceptance Testing (UAT) Results

- Customer can browse, search, and view books – Confirmed
- Customer can purchase book securely – Confirmed
- System notifies users of successful orders – Confirmed
- Clear inventory status is displayed – Confirmed

Conclusion

The software successfully passes all functional and security tests. The inventory management feature prevents overselling and fulfills all defined requirements.

Phase 6: Deployment & Release Report

1. Executive Summary

The objective of the Deployment Phase was to transition the Online Bookstore system from a local development environment to a live production environment. Following the project timeline, the application was successfully hosted on a public cloud provider, secured using SSL, and verified for public access. This phase represents the final milestone of the project lifecycle.

2. Production Environment Setup

Deliverable: Production hosting setup, domain configuration, and SSL security.

The application was deployed using a static cloud hosting service to ensure high availability, scalability, and performance.

Hosting Provider:

Netlify.

Domain Name:

<https://onlinebookstoreistinye.netlify.app/login.html>

SSL Certificate:

The website is secured via HTTPS using a TLS/SSL certificate automatically issued and managed by Let's Encrypt.

Entry Point:

The primary entry file login.html was renamed to index.html to enable automatic loading when accessing the domain.

3. CI/CD Pipeline Configuration

Deliverable: CI/CD pipeline setup.

Due to the limited scope of the project (functional demo), a manual deployment pipeline was implemented to ensure system stability.

Build:

All source files (HTML, CSS, JavaScript) were collected and validated locally.

Test:

The system was tested against all Phase 5 test cases prior to deployment.

Deploy:

The finalized project folder was uploaded to Netlify via manual drag-and-drop deployment.

Live Verification:

The production URL was checked immediately to confirm successful deployment.

4. Monitoring and Logging

Activity: Monitoring and logging implementation.

Uptime Monitoring:

System availability is monitored using the Netlify Status Dashboard, ensuring approximately 99.9% uptime.

Error Logging:

Errors are monitored through the browser console. Since the application operates using client-side logic (app.js), critical issues such as login failures or account lockouts are communicated directly to users via on-screen alerts.

5. Release Notes (Version 1.0.0)

Activity: Release documentation.

Version:

1.0.0 (Gold Master)

New Features:

- Secure user authentication with address validation and account lockout after three failed attempts.
- Real-time inventory management with automatic stock reduction.

- Dynamic catalog with star-based rating display.
- Administrative dashboard for managing users, books, and inventory.

Known Limitations:

- Data persistence relies on browser LocalStorage. Clearing the browser cache will reset stored data.

6. Rollback Plan

Activity: Rollback and recovery planning.

In case of critical production failure, the following rollback strategy is implemented:

Backup Strategy:

Each stable release is archived locally as a compressed file prior to deployment.

Rollback Procedure:

- Locate the previous stable archive.
- Re-upload the archived version to the hosting provider.

Recovery Time Objective (RTO):

The system can be restored within two minutes.

7. Final Verification (Go-Live)

Activity: Final validation and go-live approval.

The following checks were successfully completed on the live environment:

- Public URL accessibility verified.
- HTTPS security confirmed.
- End-to-end user flow tested (registration, login, purchase).

Status:

GO-LIVE SUCCEEDED.